

MCA MAJOR PROJECT PRESENTATION

Distributed Task Scheduler

A high-performance concurrent task distribution and fault-tolerant scheduling system built in Rust.

GROUP

Group 4

TEAM MEMBERS & ROLL NOS.

1. Sandipkumar Mohanbhai Bhuriya (26PG110058)
2. Khilari Sairaj Rajendra (26PG110059)
3. Gohil Bhavdeep Jayantibhai (26PG110065)

OVERVIEW

Project Objectives & Scope

Designed to decouple task submission, coordination, and distributed worker execution with extreme reliability and speed.



Asynchronous Scale

Built on Tokio async runtime for high concurrent throughput.



Fault Tolerance

Automatic heartbeat detection and dead worker task re-queuing.



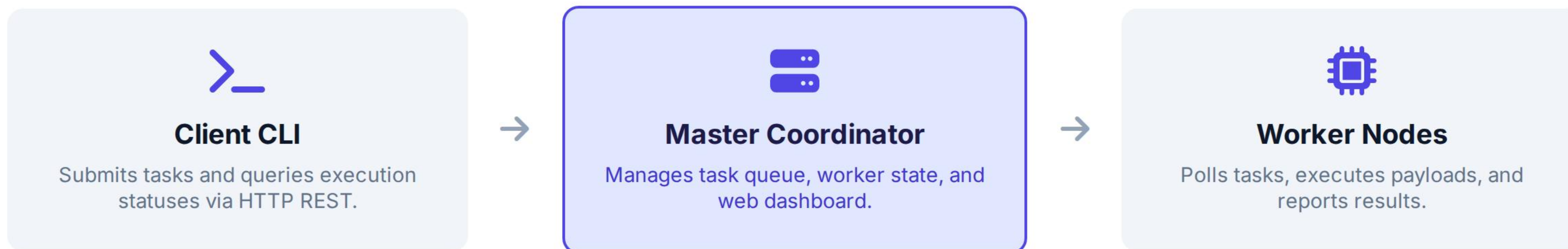
Real-Time Visibility

Embedded Tailwind CSS live control center dashboard.

DESIGN

System Architecture

A robust three-tier distributed topology connecting clients, master coordinator, and worker nodes.



CORE SERVER

Master Coordinator & Axum API

High-performance asynchronous server built with Axum routing and thread-safe shared state.

Task Lifecycle Management

Tracks states from Pending and Running to Completed or Failed.

Thread-Safe Concurrency

Utilizes Tokio Arc and RwLock for safe multi-threaded state updates.

REST Endpoints

Clean JSON APIs for task submission, worker polling, and heartbeats.

EXECUTION

Worker Node Mechanics

Independent daemon processes that register, poll, execute, and report results asynchronously.

STEP 1

Registration

Connects to master with unique worker ID.

STEP 2

Heartbeat Loop

Sends periodic liveness pings every 5 seconds.

STEP 3

Task Polling

Fetches pending items from the master queue.

STEP 4

Result Reporting

Executes payload and returns success or failure.

INTERFACE

Client CLI & Web Dashboard

Dual interface approach supporting command-line power users and visual browser monitoring.



Client CLI Tool

Provides simple subcommands: ``submit "payload"`` and ``list`` for terminal inspection.



Tailwind Web Dashboard

Auto-refreshes every 3 seconds with responsive dark-mode styling and status badges.

RELIABILITY

Fault Tolerance & Heartbeat Monitor

Ensures zero task loss even when worker nodes crash unexpectedly.

15-Second Timeout

Background daemon monitors worker heartbeat timestamps continuously.

Orphan Detection

Identifies tasks assigned to unresponsive workers instantly upon timeout.

Automatic Re-Queuing

Resets task status to Pending and clears assignment for healthy workers.

TECHNOLOGY

Technology Stack & Crates

Leveraging modern Rust ecosystem libraries for maximum memory safety and performance.

Tokio & Axum

Asynchronous runtime and web framework for fast REST routing.

Reqwest & Clap

HTTP client for worker communication and robust CLI parsing.

Serde & UUID

Fast JSON serialization and unique task ID generation.

Conclusion & Q&A

Successfully engineered and demonstrated a resilient distributed task scheduling engine in Rust.

Thank You!

Group 4 | Open for questions and live demonstration.